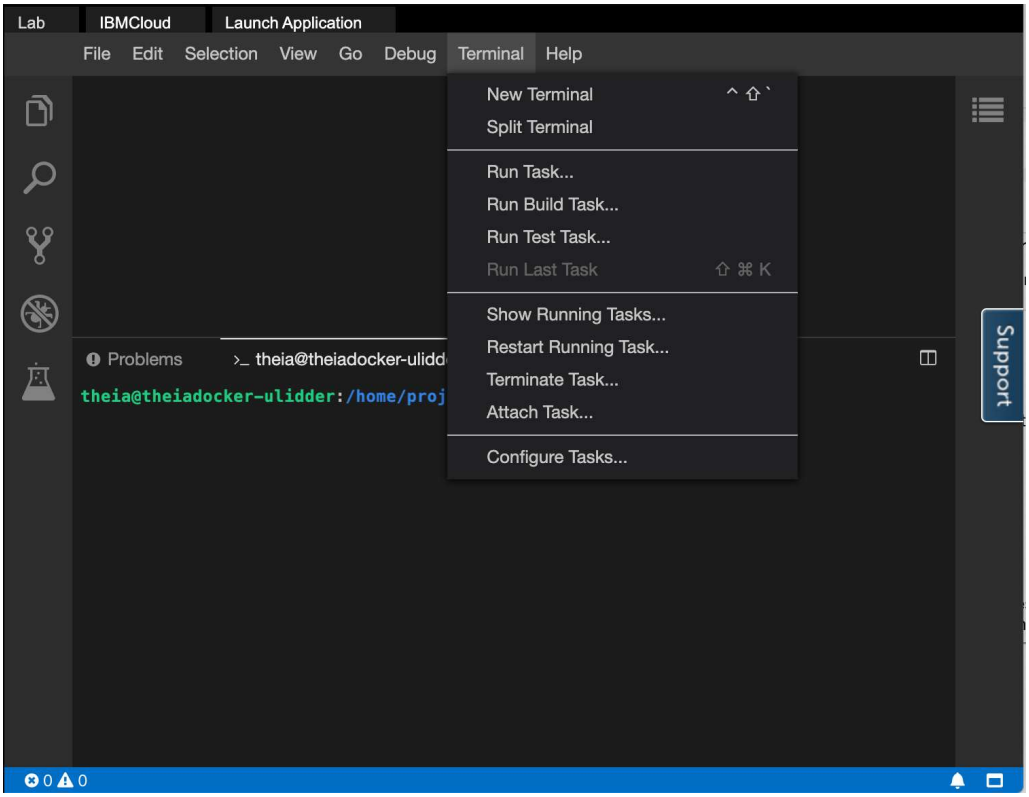
Hands-on Lab - Express Server(50 min)

Objective for Exercise:

- Create express server and run it
- Work on Middlewares with Express server
- Use middleware and JWT for authentication
- Render a static HTML page through express server

Set-up : Clone lab files

1. Open a terminal window by using the menu in the editor: Terminal > New Terminal.



2. Change to your project folder.

```
1
cd /home/project
```

Copied!

3. Check if you have the folder **lkpho-Cloud-applications-with-Node.js-and-React**

```
1
ls /home/project
```

Copied!

If you do, you can skip to step 5.

4. Clone the git repository that contains the artifacts needed for this lab, if it doesn't already exist.

```
1
git clone https://github.com/ibm-developer-skills-network/lkpho-Cloud-applications-with-Node.js-and-React.git
```

Copied!

5. Change to the directory for this lab.

```
1
cd lkpho-Cloud-applications-with-Node.js-and-React/CD220Labs/expressjs/
```

Copied!

6. List the contents of this directory to see the artifacts for this lab.

```
1
```

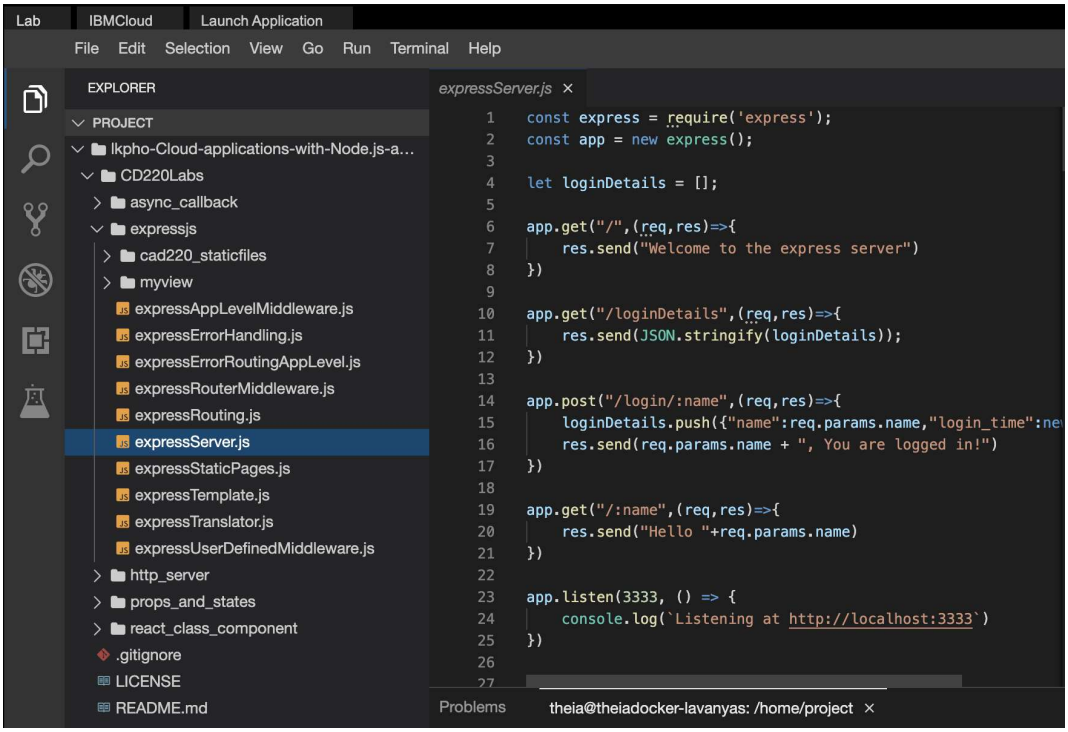
```
ls
```

Copied!

You must have a few exercise files that you will be running in the exercises.

View code, run server and connect to server through curl/browser

1. In the files explorer view expressServer.js. It would appear like this.



▶ You can also click here to view the code

This is a simple express Server which listens at port 3333, with 4 end points.

```
1
2
3
4
5
6
7
```

```
/

/loginDetails

/login/:name - POST

/:name
```

Copied!

2. In the terminal window, run the following command which will ensure that the express package is installed.

```
1
```

```
npm install --save express
```

Copied!

3. In the terminal window run the server with the following command.

```
1
```

```
node expressServer.js
```

Copied!

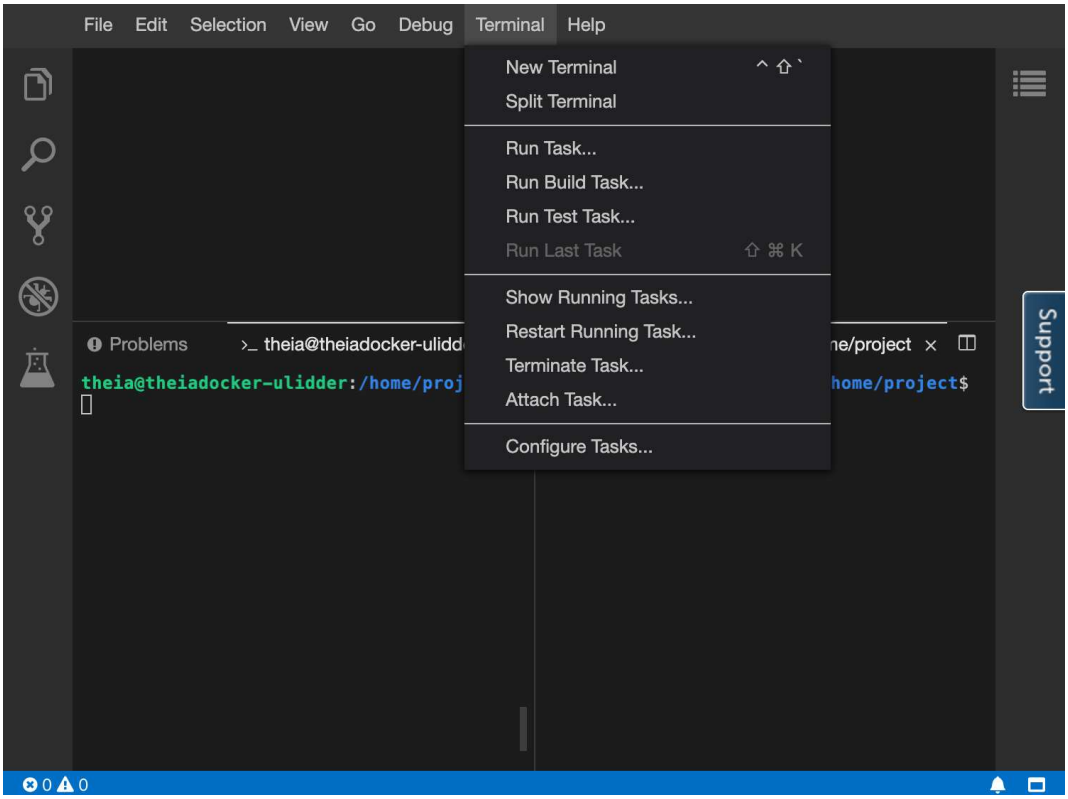
You should see output similar to this.

```
1
```

```
Listening at http://localhost:3333
```

Copied!

4. Split the terminal by clicking Terminal > Split Terminal.



5. In the second terminal window, use the `curl` command to ping the application.

```
1
curl localhost:3333
```

Copied!

You should see output similar to this.

```
1
Welcome to the express server
```

Copied!

This indicates that your app is up and running.

6. Try the other end points with the curl commands in the same terminal.

/login/:name

```
1
curl -X POST http://localhost:3333/login/Jason
```

Copied!

You should see output similar to this.

```
1
Jason, You are logged in!
```

Copied!

/:name

```
1
curl http://localhost:3333/Jason
```

Copied!

You should see output similar to this.

```
1
Hello Jason
```

Copied!

/loginDetails

```
1
curl http://localhost:3333/loginDetails
```

Copied!

You should see output similar to this.

```
1
[{"name":"Jason","login_time":"2020-11-20T06:06:56.047Z"}]
```

Copied!

7. To stop the server, go to the main command window and press Ctrl+c to stop the server.

Task 1 : Add your own end point

*Note - This is non-graded

Create a list with the names of the month. Add an end point in the code `/fetchMonth/:num` which will fetch a particular month from a list and return it to user. If the number is invalid, it should return appropriate error message.

▶ Click here, if you need help to do the task

Using Middleware

1. On the file explorer view the code expressAppLevelMiddleware.js

▶ You can click here to view the code

This server uses middleware for authentication. If the password is not pwd123 it will not allow the user to login. This server has just one end point and it takes password as query parameter.

2. Run the server.

```
1
node expressAppLevelMiddleware.js
```

Copied!

You should see output which says `Listening at http://localhost:3333`

3. In the second terminal window, use the `curl` command to ping the application.

```
1
curl localhost:3333/home
```

Copied!

You should see output which say `This user cannot login`.

4. Execute curl command passing the password parameter

```
1
curl http://localhost:3333/home?password=pwd123
```

Copied!

You should see output which say `Hello World!`.

This is because the server has a middleware which filters each request to the server to see what the password is and allows to proceed only when the password is `pwd123`.

5. To stop the server, go to the main command window and press Ctrl+c to stop the server.

Express server with Authentication

In this exercise you will learn how to build in authentication layer in your express server inorder to make the server secure. You will be using the postman tool for this lab.

1. On the file explorer view the code expressWithAuthentication.js

▶ You can click here to view the code

2. To run this application, as you may notice we use two new packages that you have not used before. Run the following command to install `jsonwebtoken` and `express-session`.

```
1
npm install --save express-session jsonwebtoken
```

Copied!

3. In this code you have one end-point, **/auth/get_message** which is allowed only for authenticated users. Run the server and try to access the end point, firstly. It should throw an error.

```
1
node expressWithAuthentication.js
```

Copied!

You should see an output which says `Listening at http://localhost:5000`.

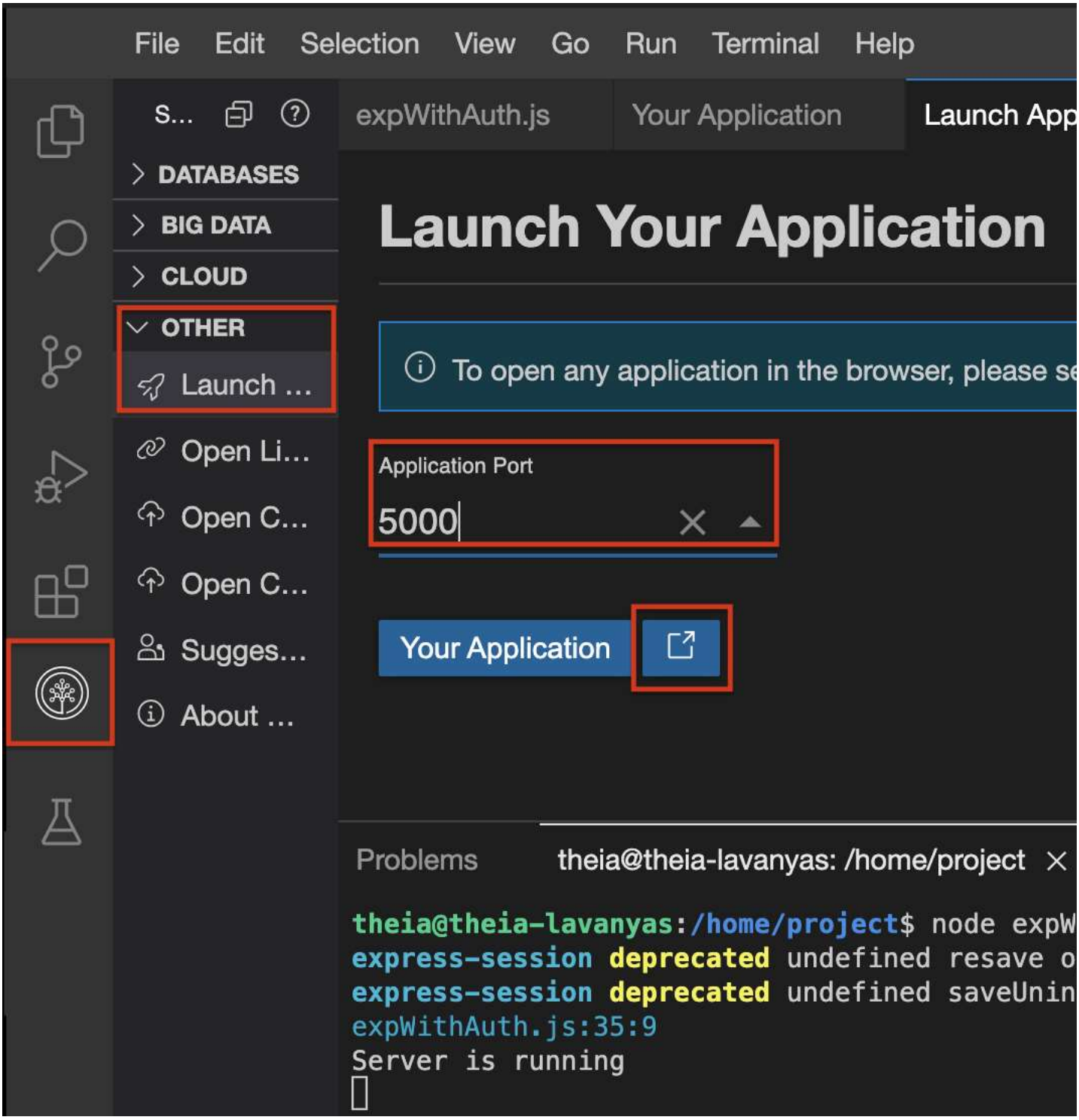
4. In the second terminal window, use the following `curl` command.

```
1
curl localhost:5000/auth/get_message
```

Copied!

You should see an output which says `{"message":"User not logged in"}`.

5. You have to register a user with a username and password and login with that username and password to be able to access the end-point. Click on the **Skills Network Toolbox** icon, choose **Others** and click **Launch Application**. Enter the port number `5000` and open the URL. It will open in a new browser window. Copy the url. Go to https://www.postman.com/. You may have to sign in if this is your first time using postman.



6. In the postman, enter the URL that you copied and suffix it with **/register**.

7. Add query parameters username and password, as shown in the following image.

POST https://lavanyas-5000.theia-1-labs-prod-misc-tools-us-east-0.proxy.c

Params Authorization Headers (7) Body Pre-request Script Tests Se

Query Params

	KEY	VALUE
☑	username	user12
☑	password	pwd12
	Key	Value

8. Set the query type to **POST** and click **send**. You will see a confirmation saying that the user has been registered.

Body Cookies (2) Headers (10) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2     "message": "User successfully registred. Now you can login"
3 }
```

9. Use the same copied url now to login with the suffix **/login**. The parameters to be passed remain the same. This is also a **POST** request. Click **send**. You will see a message confirming that you are logged in, as seen below.

POST https://lavanyas-5000.theia-1-labs-prod-misc-tools-us-east-0.proxy.c

Params Authorization Headers (7) Body Pre-request Script Tests Se

Query Params

	KEY	VALUE
☑	username	user12
☑	password	pwd12
	Key	Value

Body Cookies (2) Headers (10) Test Results

Pretty Raw Preview Visualize HTML

```
1  User successfully logged in
```

10. Now you can access the **/auth/get_message** end point and it will return the message.

GET https://lavanyas-5000.theia-1-labs-prod-misc-tools-us-east-0.proxy.c

Params Authorization Headers (6) Body Pre-request Script Tests Sett

Query Params

	KEY	VALUE
	Key	Value

Body Cookies (2) Headers (10) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2     "message": "Hello, You are an authenticated user. Congratulat
3 }
```

Routing

As the names suggests, you can route the API requests to different handlers. Usualy the handlers are logically divided on the basis of the objects they deal with.

1. On the file explorer view the code expressRouting.js

▶ You can click here to view the code

This server branches and the requests based on the end points and uses routers to handle them. All the /user endpoints are handled by userRouter and /item endpoints are handled by itemRouter.

```
1. 1
2. 2
```

```
1.  /user/:id
2.  /item/:id
```

Copied!

```
1. 1
```

```
1. node expressRouting.js
```

Copied!

You should see output which says `Listening at http://localhost:3333`.

2. In the second terminal window, use the following `curl` command.

```
1. 1
```

```
1. curl localhost:3333/item/1
```

Copied!

You should see output which says `Item 1 last enquiry Fri Nov 20 2020 15:17:46 GMT+0530 (India Standard Time)`.

3. In the second terminal window, use the following `curl` command.

```
1
curl localhost:3333/user/1
```

Copied!

You should see output which says `User 1 last successful login Fri Nov 20 2020 15:19:52 GMT+0530 (India Standard Time)`.

4. To stop the server, go to the main command window and press Ctrl+c to stop the server.

Rendering Static Pages

1. On the file explorer view the code expressStaticPages.js

▶ You can click here to view the code in expressStaticPages.js

This server, as you see doesn't have any end points. But it has a middleware which sets the directory for static files. So any file that is in the `cad220_staticfiles` directory will be accessible. The folder contains the HTML page that would be rendered.

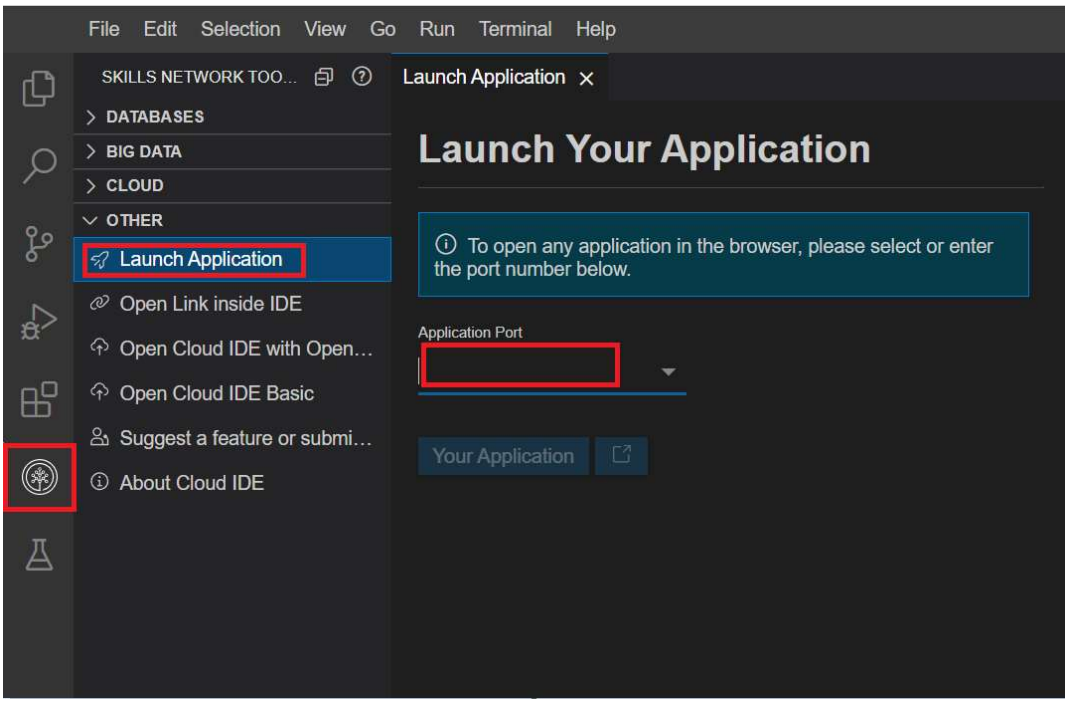
2. Run the server using the following command.

```
1
node expressStaticPages.js
```

Copied!

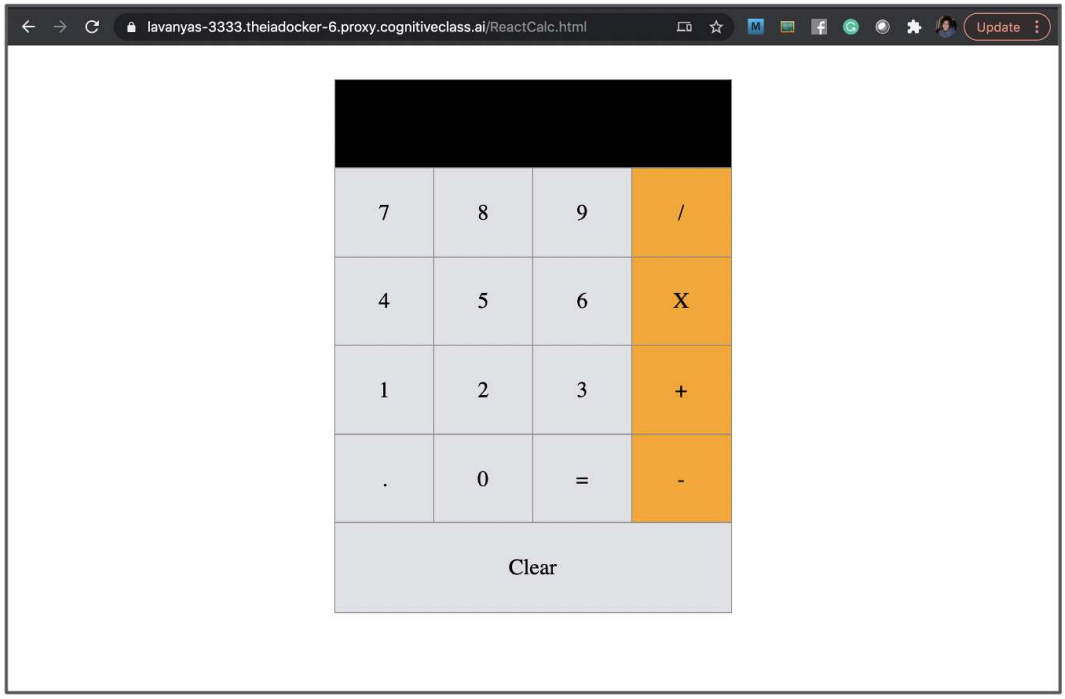
You should see output which says `Listening at http://localhost:3333`.

3. Click on the **Skills Network** button on the left, it will open the "Skills Network Toolbox". Then click the **Other** then **Launch Application**. From there you should be able to enter the port 3333 and launch.



4. Add `/ReactCalc.html` to the url in the address bar.

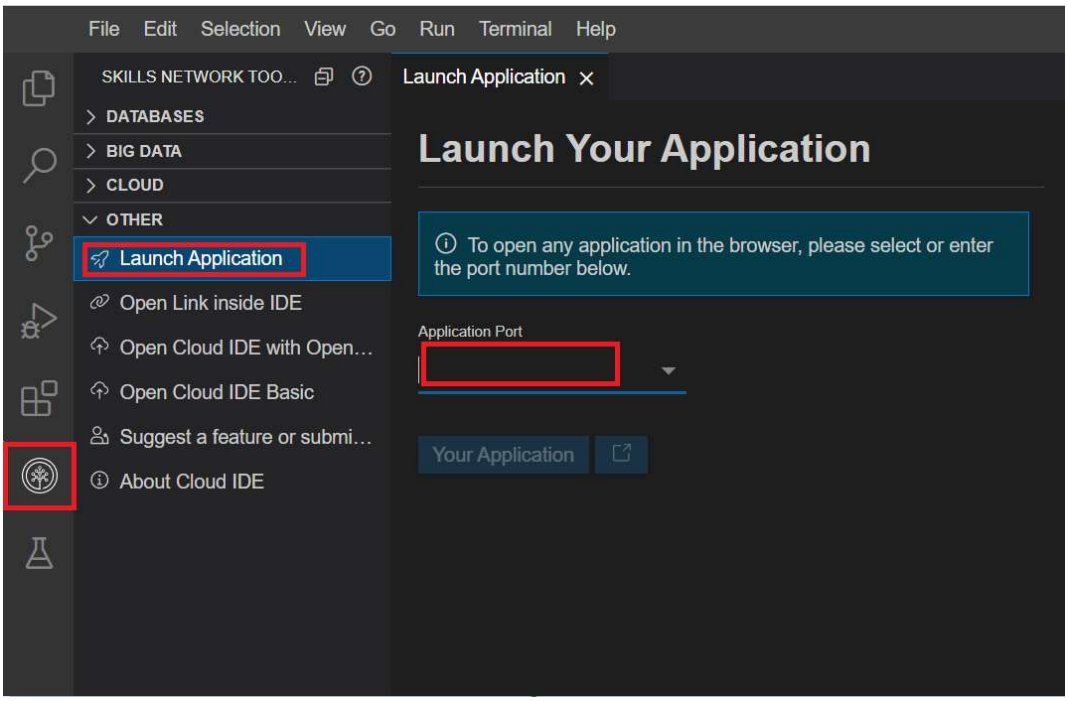
You will see th page rendered as below.



Task: Add your own static file

*Note - This is non-graded

Add a static file, an image or html file, to the directory `cad220_staticfiles` and try to access it through the `/<filename>` on the browser launched by clicking on the **Skills Network** button on the left, it will open the "Skills Network Toolbox". Then click the **Other** then **Launch Application**. From there you should be able to enter the port and launch.



Create an express server from scratch with nodemon

1. Go to the `/home/project` directory.

```
1
cd /home/project
```

Copied!

2. Create a directory named `myexpressapp` and change to that directory

```
1
2
mkdir myexpressapp
cd myexpressapp
```

Copied!

3. Now run `npm init`.

This will init the api directory to serve as a web application. Follow the prompts on the screen to complete the intialization.

- The package name by default is the name of the current folder (myexpressapp in this case). You can specify a different name if you want.
- Next it asks you for the version you want to set. The default is 1.0.0.
- It then prompts for a description where you can give a short description of what the api intends to do. You can leave it blank.
- Next we specify the entry point into the API, which by default is index.js.
- When it prompts for the author, you can give your name or leave it blank.
- License by default is ISC (Internet Systems Consortium) which means it is a permissive license that lets people do anything with your code with proper attribution and without warranty.
- It will generate the contents for your package.json, a file that keeps track of all the packages your server application needs, and asks you to check if the details are OK. Once you confirm, the details are all written on to the package.json.

4. Now run the following command to install express.

```
1
```

```
npm install express --save
```

Copied!

`--save` option ensures that the package.json is updated.

5. Now run `touch index.js` command. You will see that this file is created in the file explorer. You can use the IDE to write the code you want inside from what you have learnt in the previous exercises.

▶ Sample code has been given here

6. Make changes in `package.json` to start the server with npm start. Include `"start"` under scripts.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

```
{
  "name": "myexpressapp",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^X.X.X"
  }
}
```

Copied!

7. From the command prompt, you can now run `npm start` to run the server.

8. Now you can include other end points or make changes to the server as needed. But it can be very frustrating to stop and start the server everytime you make changes. There is a package that comes handy in this case. The package is called nodemon. Every time you make changes in the server API, it will automatically restart the server. Let' s install that in the same directory where we created our index.js. We will install and store it as a dev dependency with the –save-dev option because we want to use this only when we are running the server locally in our development environment.

```
1
```

```
npm install --save-dev nodemon
```

Copied!

9. Once nodemon is installed, we will make changes to package.json to make use of this and re-start the script when there are changes. We will include the `"start" : "nodemon index.js"` in the scripts section of our package.json. With the changes, the package.json will look like this.

```
1
2
3
4
5
```

```
6
7
8
9
10
11
12
13
14
15
16
17
18
```

```
{
  "name": "myexpressapp",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start" : "nodemon index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^X.X.X"
  },
  "devDependencies": {
    "nodemon": "^X.X.X"
  }
}
```

Copied!

At the command prompt now run `npm start` to start the web server.
Now make some change or add another endpoint returns and see if the server is restarting and changes are reflecting without having to explicitly restart. Magic!

Congratulations! You have completed the lab for express JS.

Summary

Now that you have have learnt how create and run an express server and how to use middleware, templates and routing, we will go further learn how to create clients to connect to the servers from.

Author(s)

Lavanya

Changelog

Date	Version	Changed by	Change Description
2022-11-29	4.0	K Sundararajan	Minor updates based on Beta testing feedback
2022-11-18	3.0	K Sundararajan	Instructions udpated based on Beta testing feedback
2022-10-28	2.0	Lavanya	Made changes to keep the labs upto date with new package versions
2020-11-04	1.0	Lavanya	Initial version created based videos

(C) IBM Corporation 2022. All rights reserved.